

COMP 4423: INTRODUCTION

Easy Computer Vision

Xiaoyong Wei (魏驍勇)

x1wei@polyu.edu.hk



THE HONG KONG
POLYTECHNIC UNIVERSITY
香港理工大學

Opening Minds • Shaping the Future
啟迪思維 • 成就未來



I'm interested in Computer Vision. Can you help me to prepare a learning plan for beginners?

11:28 AM ✓

 GPT-5.2

Beginner Computer Vision Learning Plan (10–12 weeks)

Prerequisites (lightweight, start in parallel)

You don't need to master these first, but you should build them as you go.

- **Python basics (1–2 weeks, then ongoing)**
Topics: functions, loops, NumPy arrays, basic plotting.
- **Math essentials (ongoing, "just in time")**
 - Linear algebra: vectors, matrices, dot product, matrix multiply
 - Probability basics: distributions, mean/variance
 - Calculus intuition: gradients (mostly for deep learning)
- **Tools**
Install: `python`, `pip/conda`, **PyTorch** (or TensorFlow), **OpenCV**, Jupyter.

Phase 1 — Image Fundamentals + OpenCV (Weeks 1–2)

Goal: Be comfortable loading, inspecting, transforming images.

Learn

- What an image is: $H \times W \times C$ arrays, color spaces (RGB/BGR), dtype, normalization
- Common ops: `resize`, `crop`, `blur`, `threshold`, `morphology`



Phase 1 — Image Fundamentals + OpenCV (Weeks 1–2)

Goal: Be comfortable loading, inspecting, transforming images.

Learn

- What an image is: $H \times W \times C$ arrays, color spaces (RGB/BGR), dtype, normalization
- Common ops: resize, crop, blur, threshold, morphology
- Gradients & edges (Sobel/Canny), histograms

Do (mini-projects)

- Build a small “image toolbox” script/notebook:
 - load → convert color → resize → denoise → edge detect → save
- Document what each transform does with before/after visuals.

Suggested resources

- OpenCV-Python tutorials (official + community)
- A short NumPy refresher if arrays feel uncomfortable



Phase 2 — Classical Computer Vision (Weeks 3–5)

Goal: Understand pre-deep-learning CV building blocks and when they still help.

Learn

- Filtering and convolution intuition
- Feature detection/descriptors: Harris, SIFT/ORB (conceptually; ORB is easiest in OpenCV)
- Feature matching and RANSAC
- Geometry basics: homography, panorama stitching (high-level)
- Optical flow basics (optional)

Do (mini-projects)

- **Panorama stitcher (simple)** using ORB + matching + homography
- **Document scanner:** detect paper contour → perspective transform → enhance



Phase 3 — Deep Learning Basics for Vision (Weeks 6–7)

Goal: Train and debug a CNN for classification; understand the training loop.

Learn

- Tensors, datasets/dataloaders
- Overfitting vs underfitting, train/val split
- Data augmentation, normalization
- Optimization basics: SGD/Adam, learning rate
- Transfer learning

Do (mini-project)

- Train a classifier on **CIFAR-10** or a small custom dataset:
 - baseline CNN from scratch
 - then transfer learning (ResNet18) and compare

Suggested resources

- PyTorch “60-minute blitz” + torchvision tutorials
- Fast.ai practical lessons (great for motivation and results)



Phase 4 — Core Vision Tasks (Weeks 8–10)

Goal: Get exposure to the “big 3” CV problems: classification, detection, segmentation.

A) Object Detection

Learn: bounding boxes, IoU, mAP basics, anchors vs anchor-free (conceptual)

Do: fine-tune a lightweight detector (e.g., YOLO family or torchvision Faster R-CNN) on a small dataset.

B) Segmentation

Learn: semantic vs instance segmentation, masks, Dice/IoU loss intuition

Do: train a U-Net-style model on a small segmentation dataset (even 200–1000 images is enough to learn).

C) Practical deployment habits (light)

- Exporting models (TorchScript/ONNX basics)
- Inference speed, batch vs single image, resizing strategy



Phase 5 — Capstone Project (Weeks 11–12)

Goal: Build one portfolio-worthy application end-to-end.

Pick one:

1. **Traffic sign / product recognition** (classification + augmentation + deployment notebook)
2. **People/vehicle counter** (detection + tracking like SORT/DeepSORT)
3. **Medical-style segmentation** (U-Net on public dataset)
4. **OCR pipeline** (classical preprocessing + deep text recognition if time)

Deliverables:

- Clean repo + README + sample images + metrics
 - A short report: what worked, what didn't, what you'd improve
-

Weekly time budget (realistic)

- **5–8 hrs/week:** steady progress
- **10–15 hrs/week:** you'll finish the full plan comfortably

A good pattern:

- 60% coding/projects
- 30% reading/watching
- 10% writing notes + documenting experiments



What to learn next (after the plan)

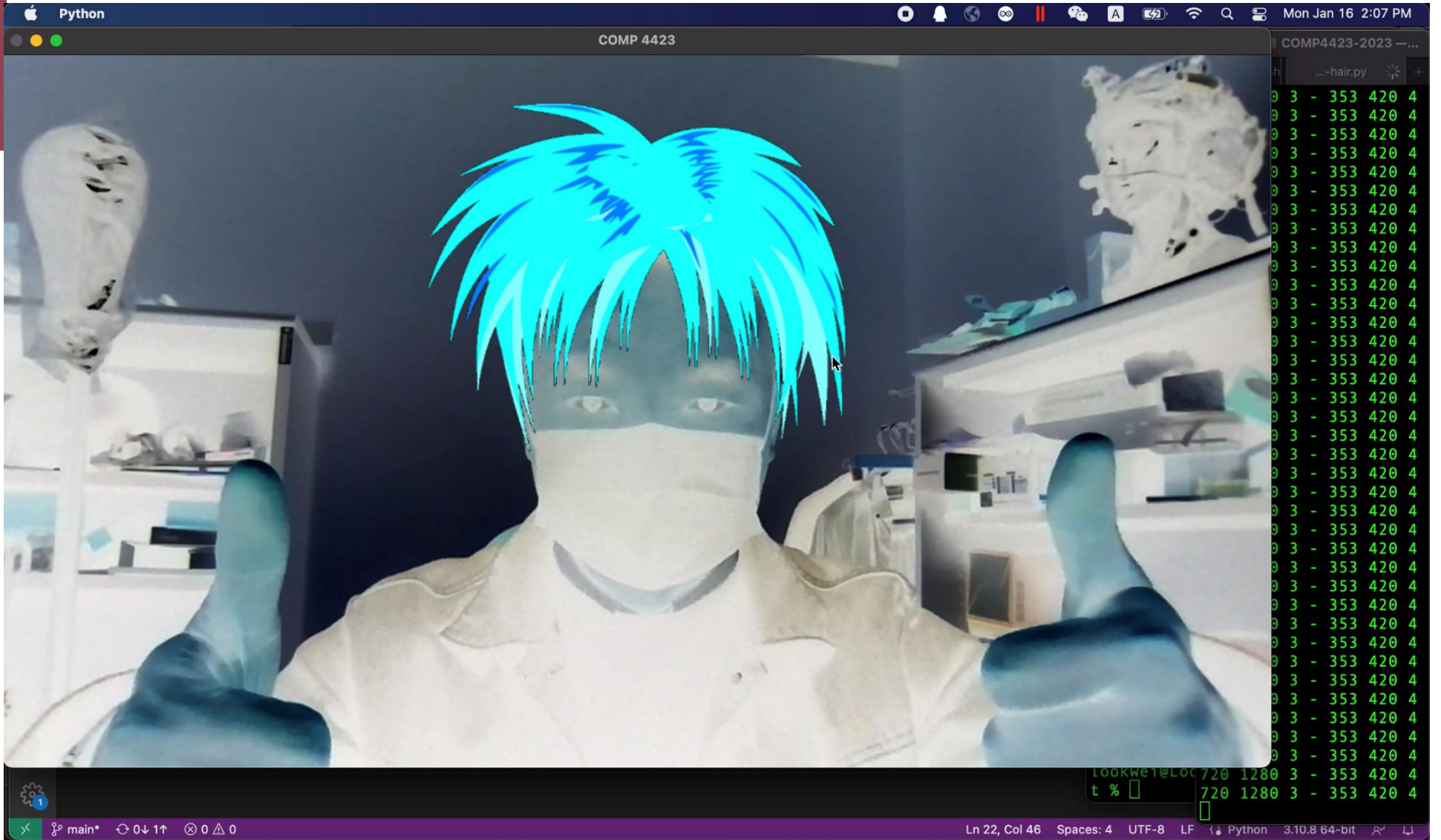
Depending on interest:

- **3D Vision:** stereo, depth, SLAM
- **Vision Transformers:** ViT, DeiT, CLIP embeddings
- **Self-supervised learning:** contrastive methods
- **Multimodal:** vision + language (captioning, VQA)

Quick questions (so I can tailor it)

1. How many hours per week can you spend?
2. Do you prefer **PyTorch** or **TensorFlow**, or no preference?
3. Are you aiming for **research**, **industry/job**, or **hobby projects**?
4. Do you have a GPU available (local or cloud)?

Answer those and I'll turn this into a week-by-week schedule with specific exercises and datasets that match your goals.



Methods will be introduced in this lecture



Outline

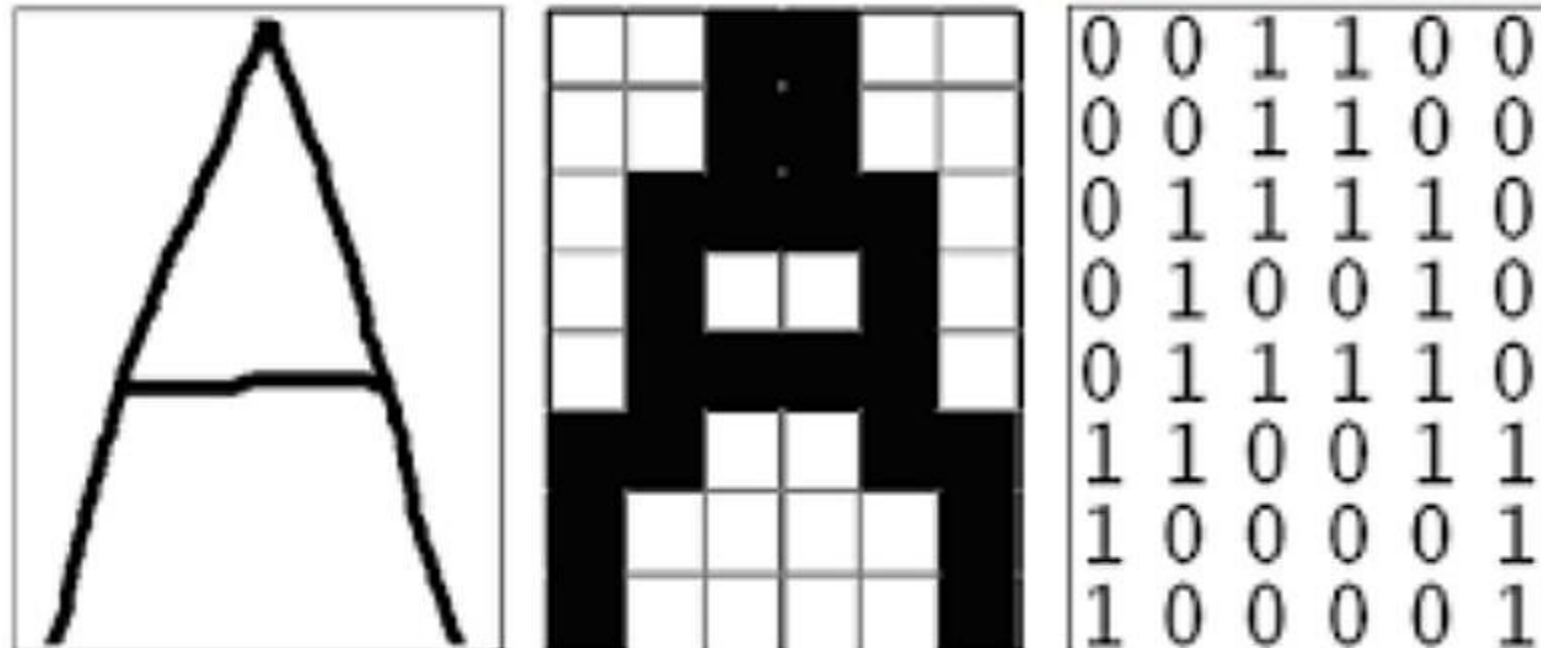
- How Human/Computers see images
- Display the images
- Play the images (colors, sizes, rotations)
- Play the videos (files, webcam)



What is Computer Vision?



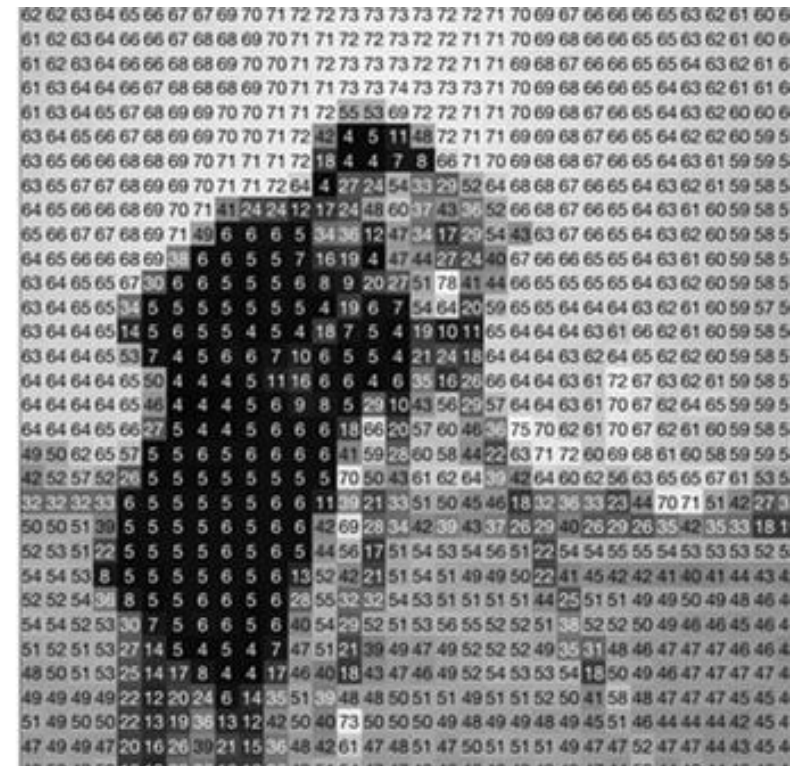
What human/computers see



<https://santanderglobaltech.com/en/computer-vision-vs-human-vision-this-is-how-computers-see/>

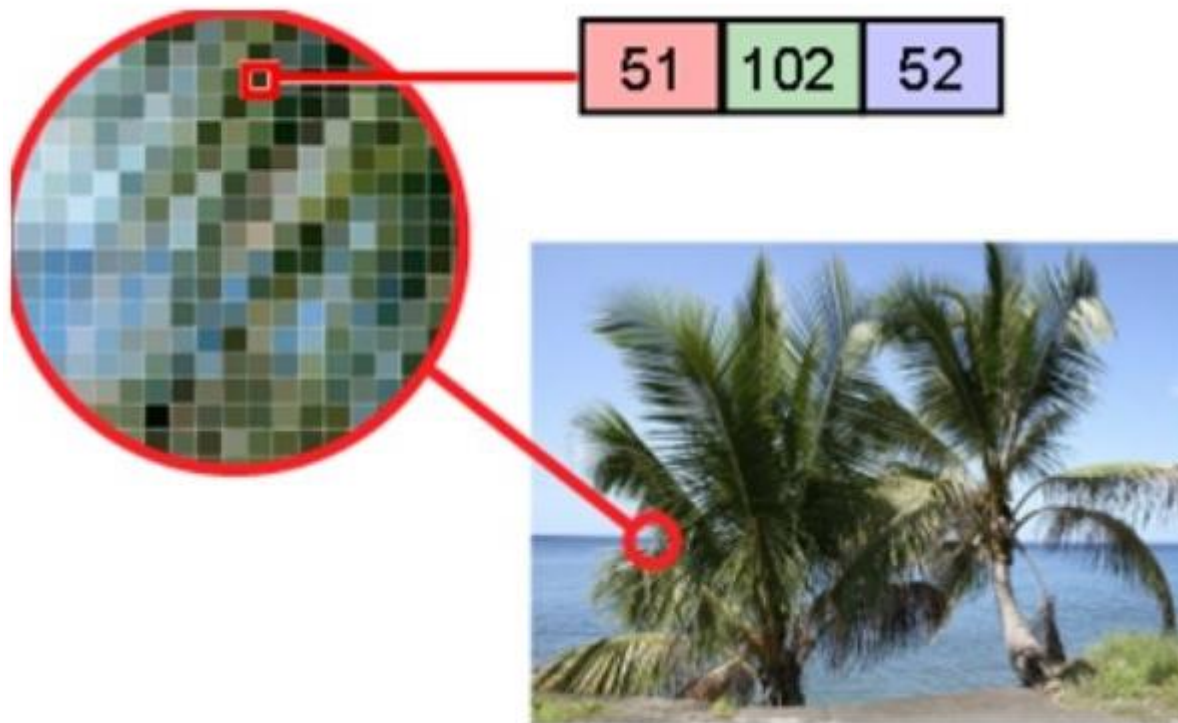


What human/computers see





What human/computers see



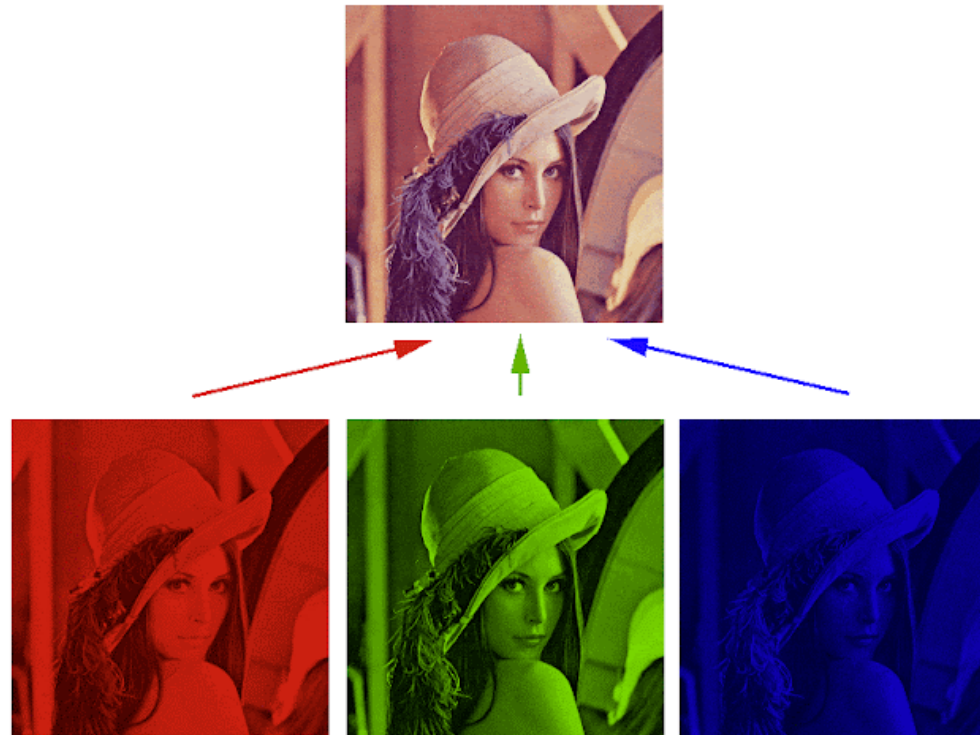
RGB



<https://santanderglobaltech.com/en/computer-vision-vs-human-vision-this-is-how-computers-see/>



What human/computers see



<http://spsprashanth.blogspot.com/2016/08/rgb.html>



TALK
IS CHEAP SHOW ME
THE CODE



Let's play it!



Get yourself a Google CoLab

- 1. You need a Google Account
- 2. Sign in at https://colab.research.google.com/?utm_source=scs-index
- 3. You're all set!



Load an image



.bmp

Device-Independent Bitmap (BMP)

.jpg or .jpeg

Joint Photographic Experts Group (JPEG)

.tif or .tiff

Tagged Image File Format (TIFF)



Load an image

 image.processing1.ipynb ☆
File Edit View Insert Runtime Tools Help [All changes saved](#)

+ Code + Text



Image Processing 1: Let's play with the images

1. Load an image

There are several ways to load images. We show two examples of *PIL* and *skimage.io* here. Besides, to load an image with the url, help of *urllib.request*.

```
from PIL import Image
import urllib.request
import skimage.io

# save the image from the url
urllib.request.urlretrieve('https://upload.wikimedia.org/wikipedia/en/7/7d/Lenna_%28test_image%29.png',
                           'lenna.png')

# load the image using PIL.Image
img_lenna = Image.open('lenna.png')
display(img_lenna)

# load the image using skimage.io
img_lenna2 = skimage.io.imread(fname='lenna.png')
skimage.io.imshow(img_lenna2)
```

Import the libraries. Recall the *include* in C or *using* in C#

Call a function



Load an image

 image.processing1.ipynb ☆
File Edit View Insert Runtime Tools Help [All changes saved](#)

+ Code + Text

Image Processing 1: Let's play with the images

1. Load an image

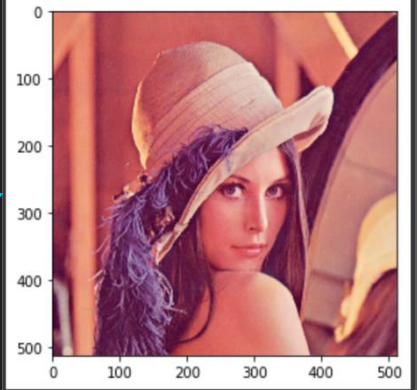
There are several ways to load images. We show two examples of *PIL* and *skimage.io* here. Besides, to load an image with the url, help of *urllib.request*.

```
from PIL import Image
import urllib.request
import skimage.io

# save the image from the url
urllib.request.urlretrieve('https://upload.wikimedia.org/wikipedia/en/7/7d/Lenna_%28test_image%29.png',
                           'lenna.png')

# load the image using PIL.Image
img_lenna = Image.open('lenna.png')
display(img_lenna)

# load the image using skimage.io
img_lenna2 = skimage.io.imread(fname='lenna.png')
skimage.io.imshow(img_lenna2)
```





See what it is

2. See what it is

```
[43] # check out the image
      print('Format:\t'+img_lenna.format)
      print('Size:\t'+"%s"%(img_lenna.size,))
      print('Mode:\t'+img_lenna.mode)

      # check out the image as arrays
      import numpy as np
      im = np.array(img_lenna)

      print('Array Type:\t'+str(type(im)))
      print('Data Type:\t'+str(im.dtype))
      print('Array Shape:\t'+"%s"%(im.shape,))
```

```
Format: PNG
Size:   (512, 512)
Mode:   RGB
Array Type: <class 'numpy.ndarray'>
Data Type: uint8
Array Shape: (512, 512, 3)
```



See what it is

```
# check out channels  
R_img=Image.fromarray(im[:, :, 0])  
display(R_img)  
G_img=Image.fromarray(im[:, :, 1])  
display(G_img)  
B_img=Image.fromarray(im[:, :, 2])  
display(B_img)
```





What information do these
channels tell?



The R channels are dominating. It results in a **warm** color image.



+



+





Can we make it a cool color?



We can see from above that the R channel is dominating, which results in a warm color image. Can we make it a cool color? It can be done by simply "inverse" the value of each pixel within the range of $[0, 255]$. For example, for a value x (e.g., 30), its inverted value is $(255-x)$ (e.g., $255-30 = 225$).



```
cool_img=Image.fromarray(255-im)  
display(cool_img)
```

Handwritten red annotations: $0 \rightarrow 255$, $1 \rightarrow 254$, $250 \rightarrow 5$.



Handwritten red annotation: 12.

Handwritten red annotation: 6.5x 1.3.





Other solutions?



It's too large to fit into my small screen.

Can you make it smaller?



Resize an image

Sure. We can resize an image (let's call it the source image hereafter) by generating a target image with pixels selected from the source. For example, to reduce the size by 50%, we can select one from two neighbour pixels into the target image.



```
from numpy.core.fromnumeric import shape
cool_array=255-im
width_new,height_new=int(im.shape[0]/2),int(im.shape[1]/2)

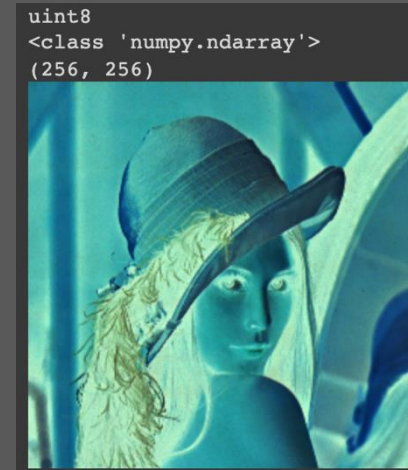
cool_array_small=np.ndarray((width_new,height_new,3),dtype=np.uint8)
for i in range(0,width_new):
    for j in range(0,height_new):
        cool_array_small[i,j,:]=cool_array[2*i,2*j,:]

cool_img_small=Image.fromarray(cool_array_small)
print(cool_array_small.dtype)
print(type(cool_array_small))
print(cool_img_small.size)
display(cool_img_small)
```



Resize an image

```
cool_array_small=np.ndarray((width_new,height_new,3),dtype=np.uint8)
for i in range(0,width_new):
    for j in range(0,height_new):
        cool_array_small[i,j,:]=cool_array[2*i,2*j,:]
```





Similarly, we can enlarge the image by duplicating the pixels to its neighbours in the target.
Let's double the size of the source.



✓
2s

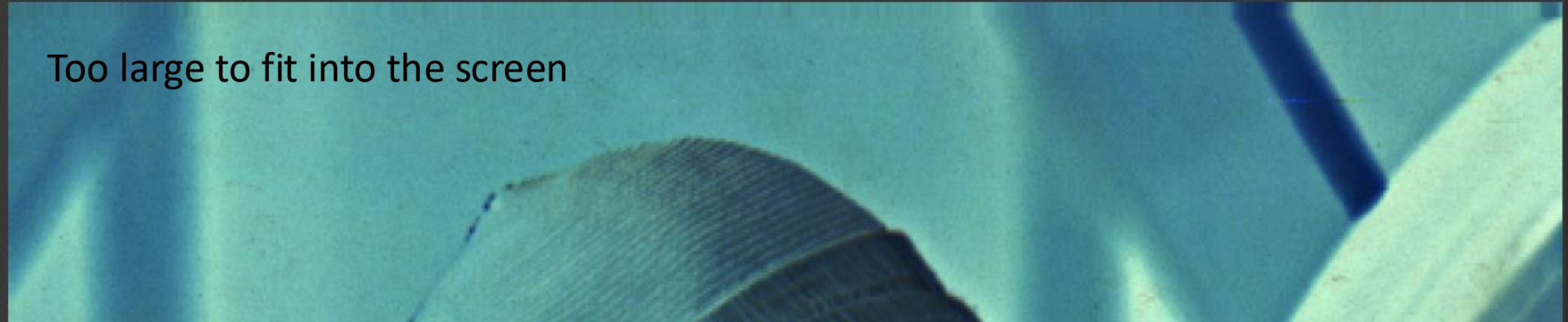
```
width_lg,height_lg=int(im.shape[0]*2),int(im.shape[1]*2)

cool_array_large=np.ndarray((width_lg,height_lg,3),dtype=np.uint8)
for i in range(0,width_lg):
    for j in range(0,height_lg):
        cool_array_large[i,j,:]=cool_array[int(i/2),int(j/2),:]

cool_img_large=Image.fromarray(cool_array_large)
print(cool_array_large.dtype)
print(type(cool_array_large))
print(cool_img_large.size)
display(cool_img_large)
```

```
uint8
<class 'numpy.ndarray'>
(1024, 1024)
```

Too large to fit into the screen



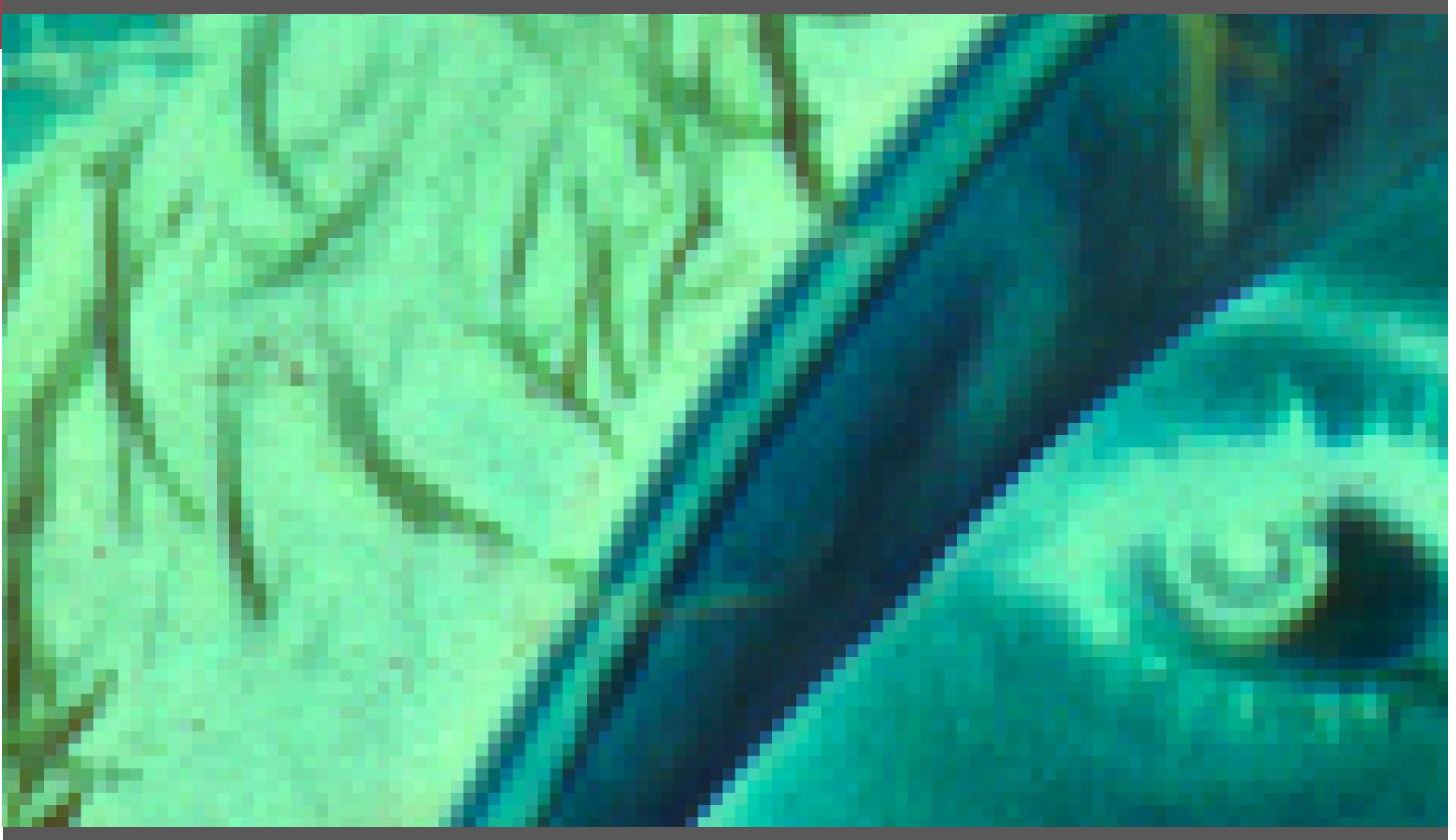


It's cool! Can we enlarge an image
into any size?

Let's try enlarge it by 10 times using
the same code and see what
happens.



Enlarge the image by 10 times





What happened?





Image Interpolation

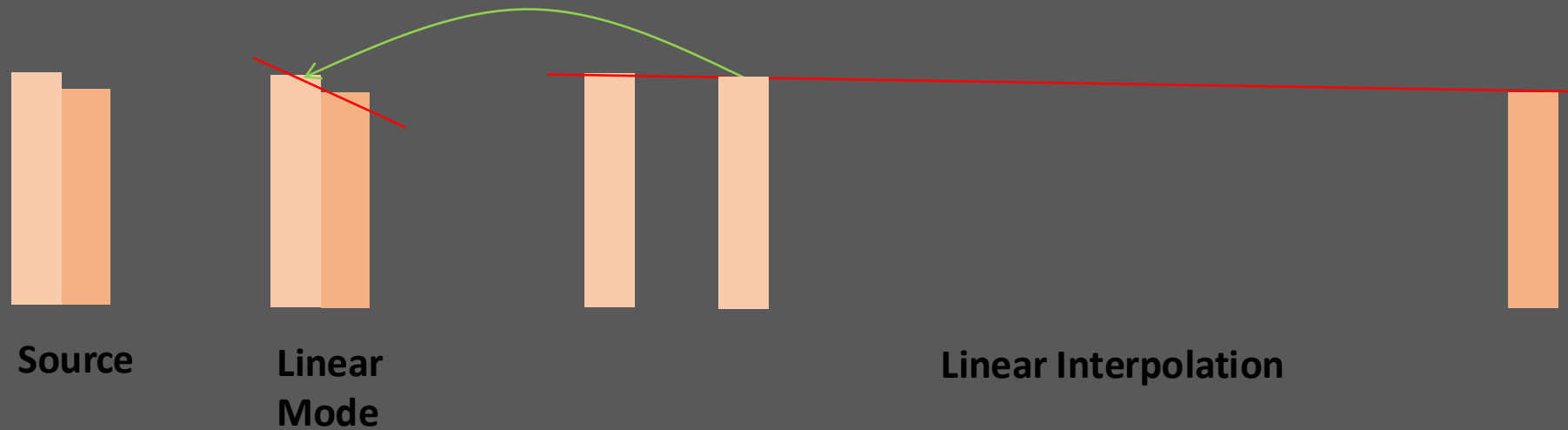
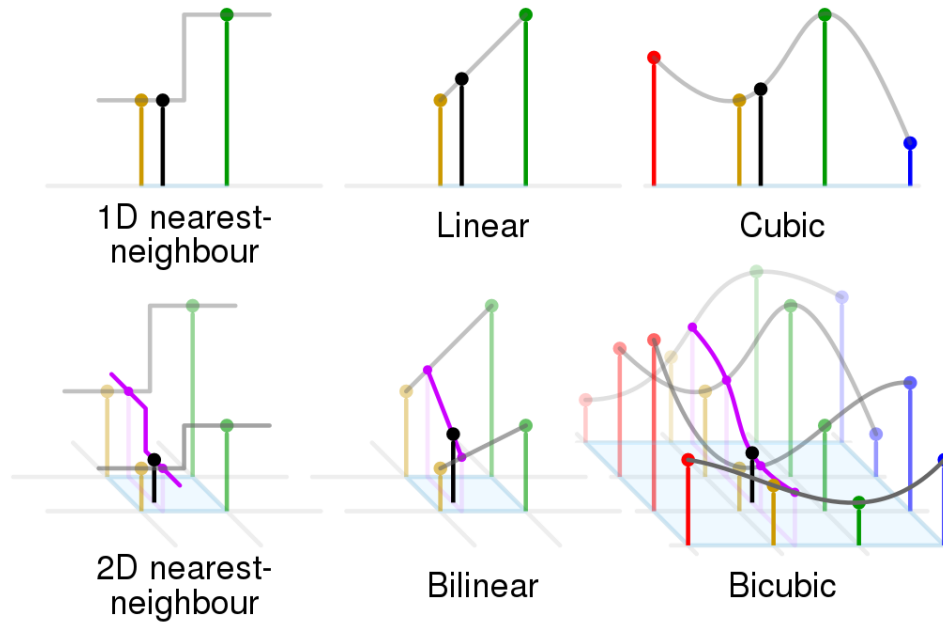
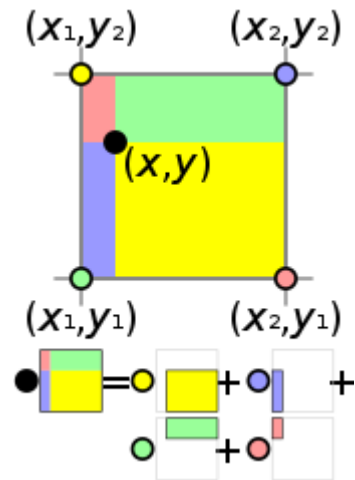




Image Interpolations



https://en.wikipedia.org/wiki/Bicubic_interpolation



Great!
What if I want to watch it while
lying in bed?





Image Rotation

$$x_2 = \cos(\theta) * (x_1 - x_0) - \sin(\theta) * (y_1 - y_0) + x_0$$

$$y_2 = \sin(\theta) * (x_1 - x_0) + \cos(\theta) * (y_1 - y_0) + y_0$$

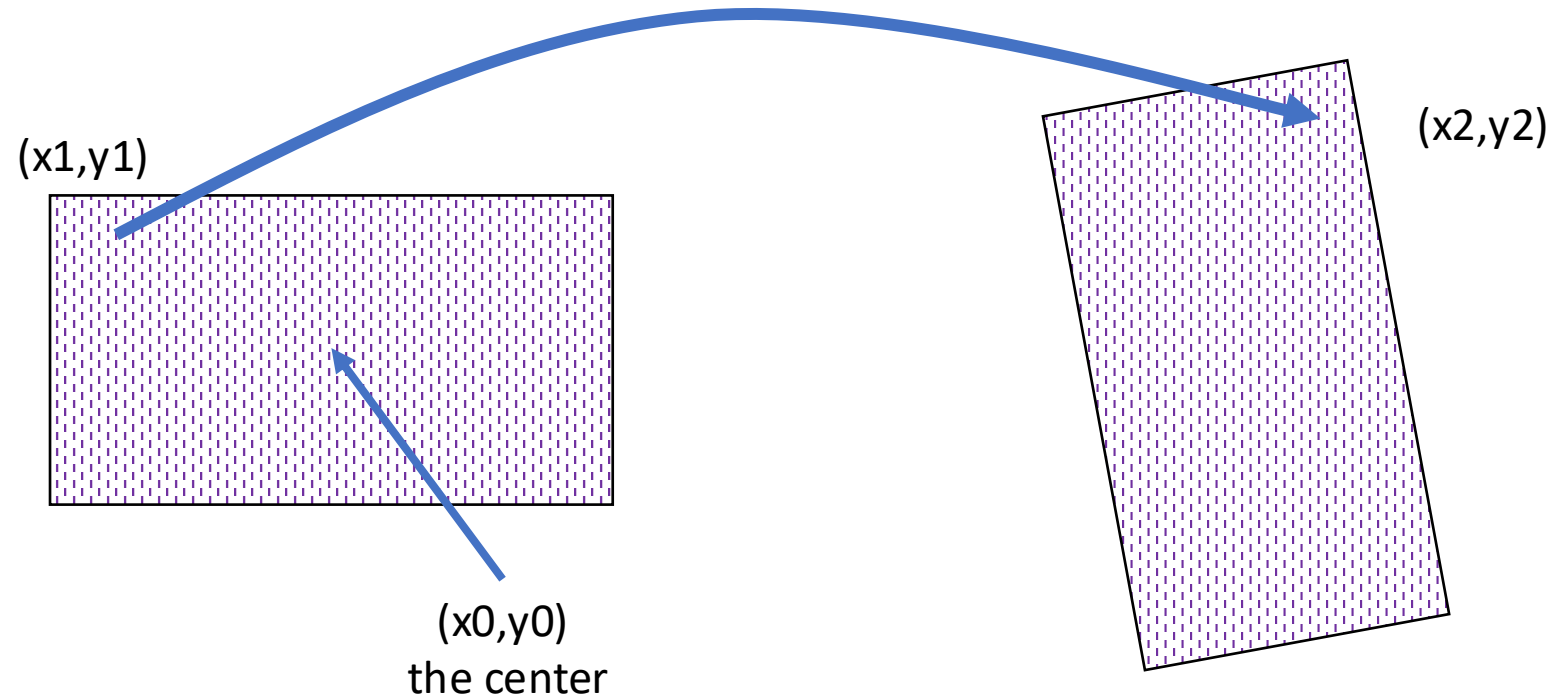




Image Rotation

Great! Let's take a step further. Say, we want to watch the image when we're lying on the bed, and it would be better to rotate the image clockwise by 90 degree so that we don't have to rotate our screen.



```
width_cw90,height_cw90=cool_array.shape[1],cool_array.shape[0]

cool_array_cw90=np.ndarray((width_cw90,height_cw90,3),dtype=np.uint8)
for i in range(0,width_cw90):
    for j in range(0,height_cw90):
        y0,x0=int(height_cw90/2),int(width_cw90/2)
        y1,x1=j,i
        y2,x2=min((x1-x0)+y0,width_cw90-1),min(-(y1-y0)+x0,height_cw90-1)
        cool_array_cw90[i,j,:]=cool_array[x2,y2,:]

cool_img_cw90=Image.fromarray(cool_array_cw90)
print(cool_img_cw90.size)
display(cool_img_cw90)
```



```
cool_array_cw90=np.ndarray((width_cw90,height_cw90,3),dtype=np.uint8)
for i in range(0,width_cw90):
    for j in range(0,height_cw90):
        y0,x0=int(height_cw90/2),int(width_cw90/2)
        y1,x1=j,i
        y2,x2=min((x1-x0)+y0,width_cw90-1),min(-(y1-y0)+x0,height_cw90-1)
        cool_array_cw90[i,j,:]=cool_array[x2,y2,:]
```





Image Rotation

To **rotate** 45° about the origin, we apply the matrix

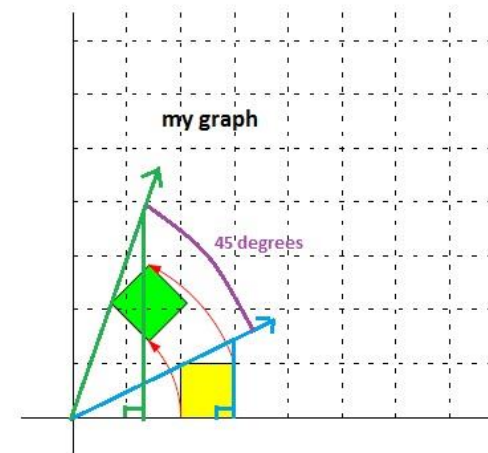
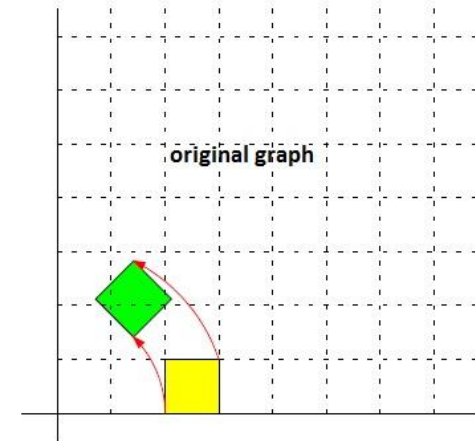
$$\begin{pmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{pmatrix} = \frac{\sqrt{2}}{2} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$$

Note: $\frac{\sqrt{2}}{2} = \cos 45^\circ = \sin 45^\circ$, so this is the same as

$$\begin{pmatrix} \cos 45^\circ & -\sin 45^\circ \\ \sin 45^\circ & \cos 45^\circ \end{pmatrix}$$

Counter Clockwise $\begin{pmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{pmatrix}$

Clockwise $\begin{pmatrix} \cos \phi & \sin \phi \\ -\sin \phi & \cos \phi \end{pmatrix}$





Geometric Manipulations as Matrix Transformations

$$\begin{bmatrix} x_t \\ y_t \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ or } \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \text{ or } \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x_s \\ y_s \\ 1 \end{bmatrix}$$

Rotation

Translation

Scaling



Congratulations!

You just grasped the fundamentals of
image processing.

But ...



There are much easier ways ...



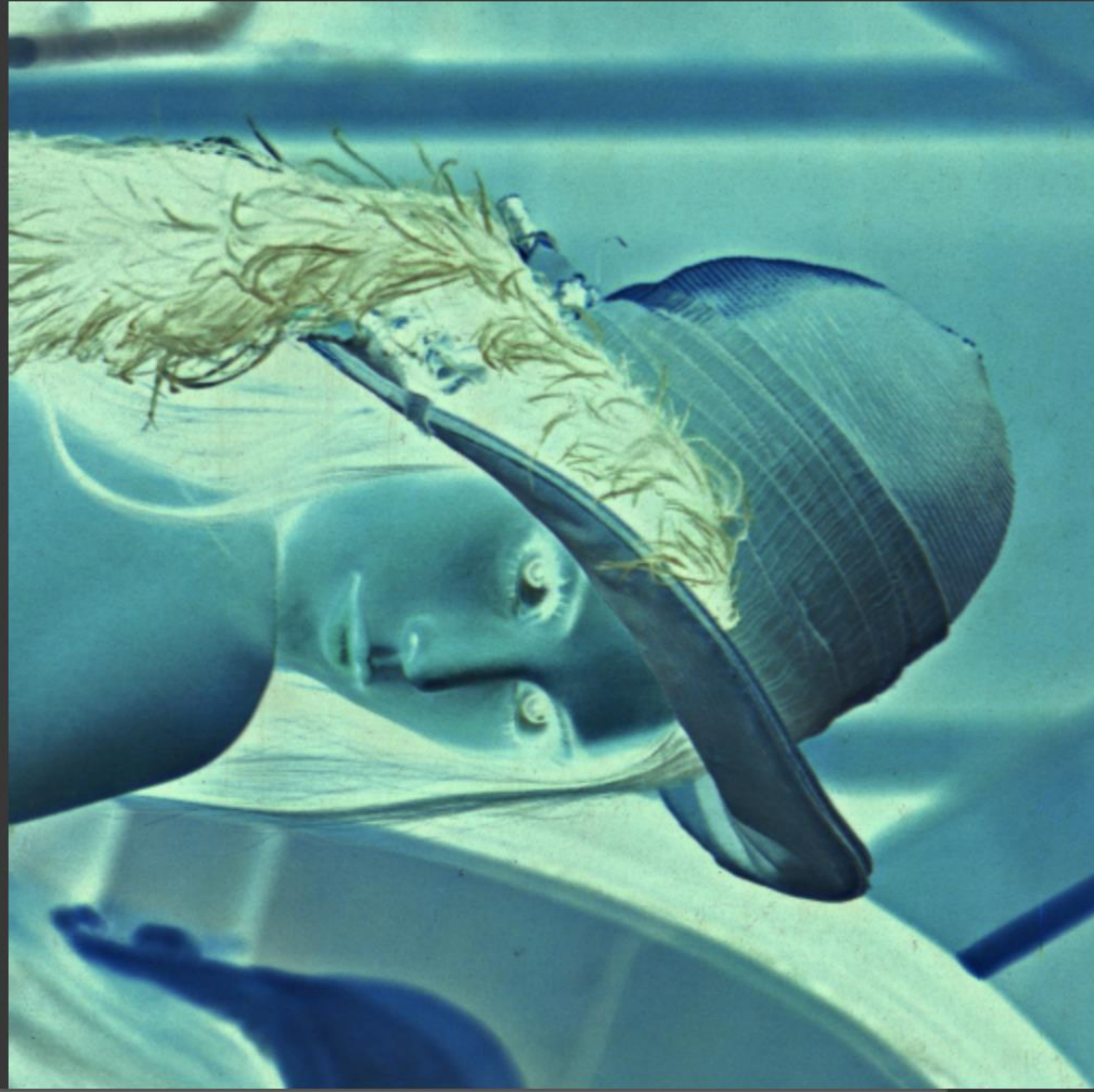
✓
1s



```
import cv2 #import OpenCV
```

```
img_rotate_90_clockwise = cv2.rotate(cool_array, cv2.ROTATE_90_CLOCKWISE)  
cool_img_cw90=Image.fromarray(img_rotate_90_clockwise)  
print(cool_img_cw90.size)  
display(cool_img_cw90)
```

(512, 512)





✓
1s



```
img_flip_ud = cv2.flip(cool_array, 0)
cool_img_new=Image.fromarray(img_flip_ud)
print(cool_img_new.size)
display(cool_img_new)
```



(512, 512)





```
✓ 0s  # convert the image into gray
im_gray = cv2.cvtColor(cool_array, cv2.COLOR_BGR2GRAY)
# binarize the image using a threshold
th, im_gray_th_otsu = cv2.threshold(im_gray, 128, 192, cv2.THRESH_OTSU)
cool_img_new=Image.fromarray(im_gray_th_otsu)
print(cool_img_new.size)
display(cool_img_new)
```

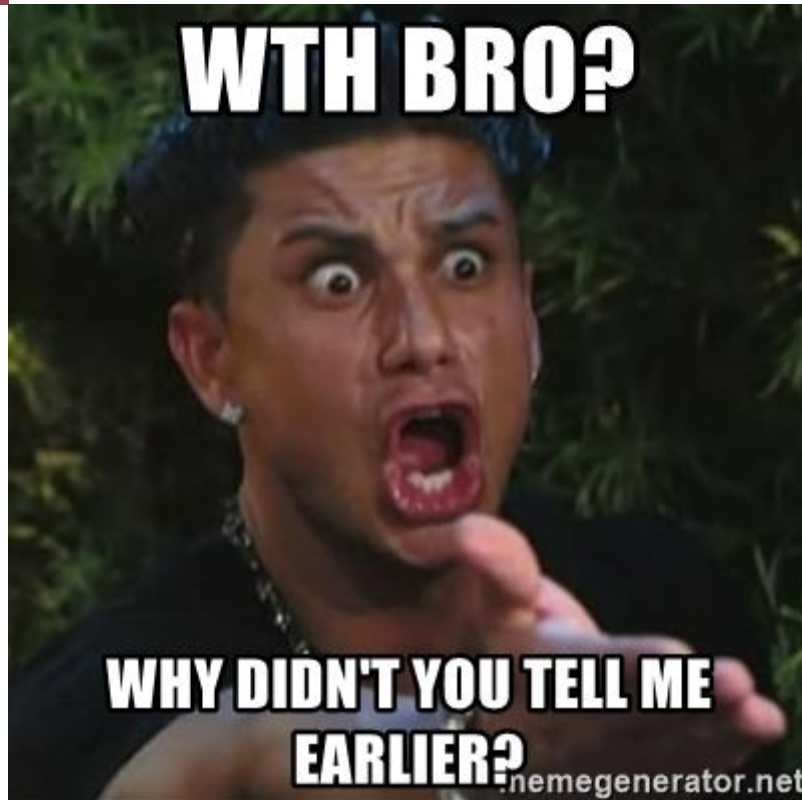
(512, 512)





```
cool_img_lg=cool_img.resize((800,300),Image.BICUBIC)  
display(cool_img_lg)
```





Why didn't you
tell us earlier?



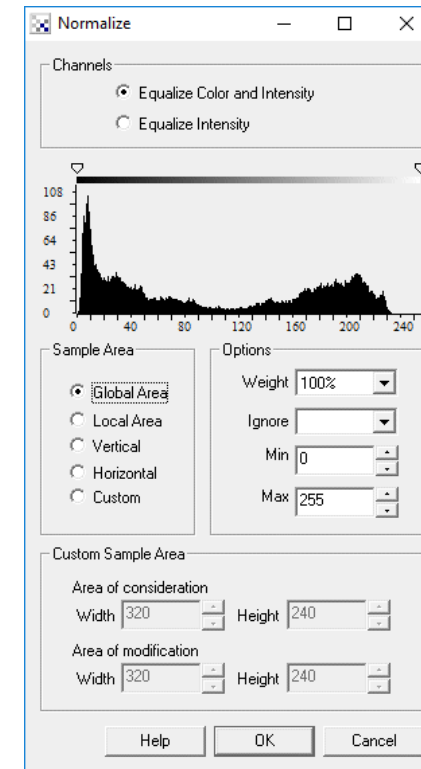
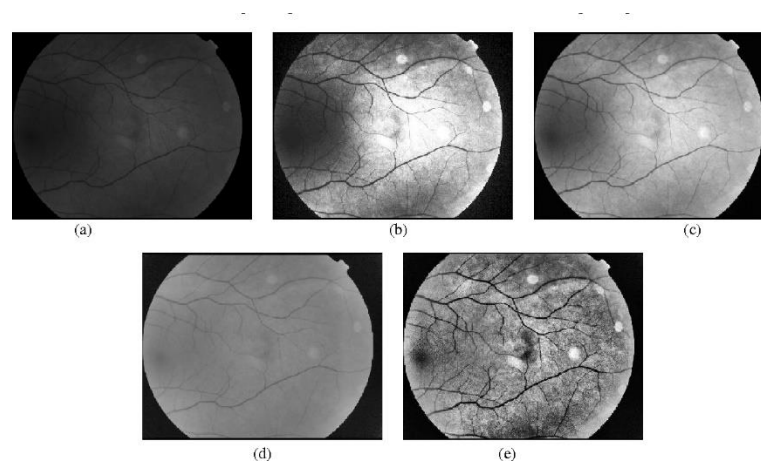
Why would I have to learn those?
Where do the fancy stuff in the
Lecture 1 fit in?



Image preprocessing are the steps taken before the model training.



Illumination Normalization





Sensor Gap and Color Normalization

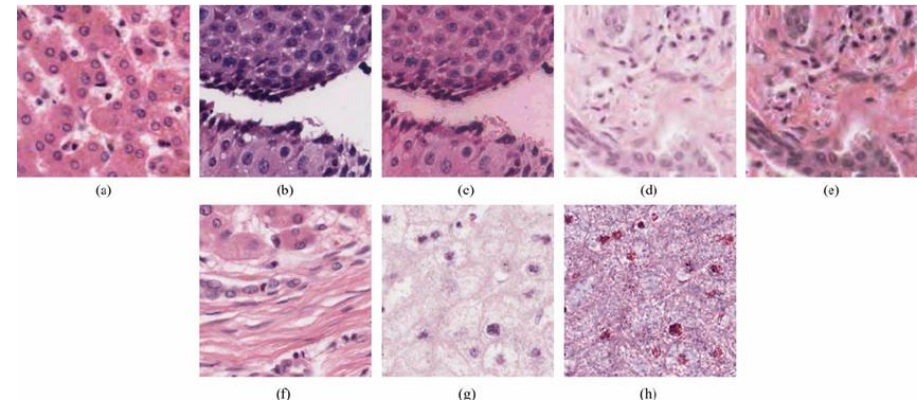
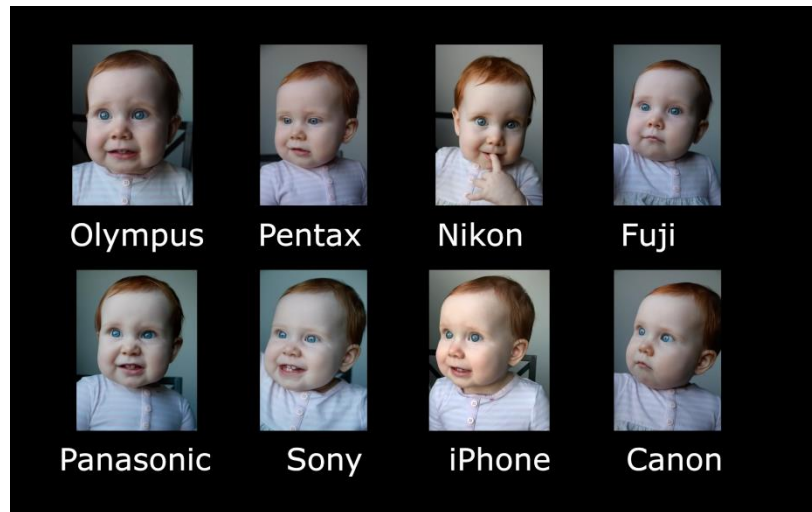




Image Registration

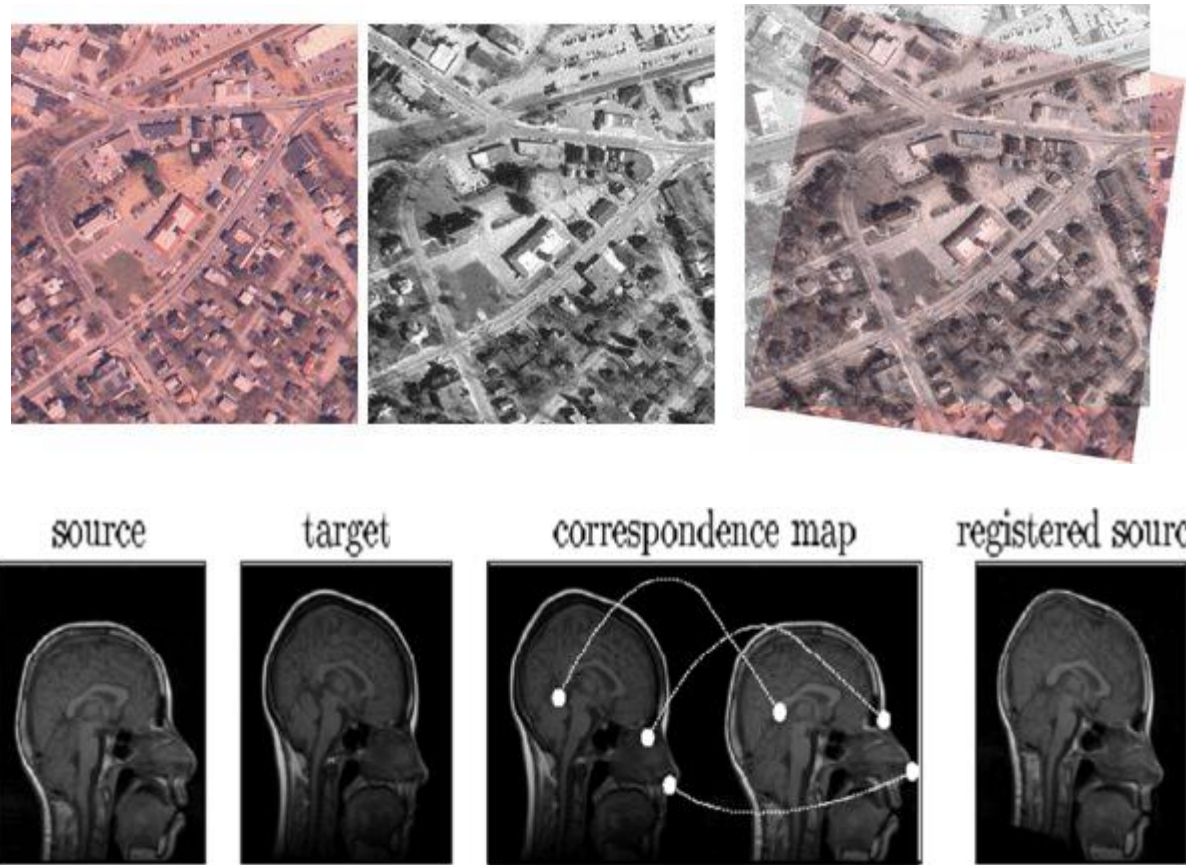


Figure 1.2: Image Registration Process



Let's play with videos



Load video stream (webcam)

```
import cv2

# declare a video capture object
vid = cv2.VideoCapture(0)

while True:
    # capture the video frame by frame
    ret, frame_raw = vid.read()
    frame = cv2.flip(frame_raw, 1) # optional
    # display the resulting frame
    cv2.imshow('COMP 4423', frame)

    # quit when 'q' is pressed
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

# release the cap object
vid.release()
# destroy all the windows
cv2.destroyAllWindows()
```



Invert the images (frames)

```
import cv2

# declare a video capture object
vid = cv2.VideoCapture(0)
inversion_on=False

while True:
    # capture the video frame by frame
    ret, frame_raw = vid.read()
    frame = cv2.flip(frame_raw, 1) # optional
    # get dimensions of the frame
    h, w, c = frame.shape
    print(h,w,c)
    # display the resulting frame
    cv2.imshow('COMP 4423', 255-frame if inversion_on else frame)

    key=cv2.waitKey(1) & 0xFF
    # quit when 'q' is pressed
    if key== ord('q'):
        break
    if key==ord('i'):
        inversion_on=not inversion_on

# release the cap object
vid.release()
# destroy all the windows
cv2.destroyAllWindows()
```



Put the hair on

```
1 import cv2,numpy as np
2
3 # declare a video capture object
4 vid = cv2.VideoCapture(0)
5 inversion_on=False
6
7 # load the redhair image
8 im_readhair = cv2.imread('redhair.png', cv2.IMREAD_UNCHANGED)
9 hh,wh,ch=im_readhair.shape
10 # resize the hair image
11 im_readhair = cv2.resize(im_readhair, (int(wh*0.7),int(hh*0.7)), interpolation = cv2.INTER_AREA)
12 # find the indices of non-transparent pixels
13 non_trans=np.argwhere(im_readhair[:, :, 3]>0)
14
15 h2,w2,c2=im_readhair.shape
16 alpha = im_readhair[:, :, 3]
17 hair_on=False
18
19 while True:
20     # capture the video frame by frame
21     ret, frame_raw = vid.read()
22     frame = cv2.flip(frame_raw, 1) # optional
23     # get dimensions of the frame
24     h, w, c = frame.shape
25     print(h,w,c, '-',h2,w2,c2)
26
```





Put the hair on (cont.)

```
21 ret, frame_raw = vid.read()
22 frame = cv2.flip(frame_raw, 1) # optional
23 # get dimensions of the frame
24 h, w, c = frame.shape
25 print(h,w,c, '-', h2,w2,c2)
26
27 # put the hair on
28 if hair_on:
29     top,left=50,int((w-w2)/2)
30     #frame[top:top+h2,left:left+w2,:]=im_readhair[:, :, 0:3]
31     frame[top+non_trans[:,0],left+non_trans[:,1],:]=im_readhair[non_trans[:,0],non_trans[:,1],0:3]
32
33 # display the resulting frame
34 cv2.imshow('COMP 4423', 255-frame if inversion_on else frame)
35
36 key=cv2.waitKey(1) & 0xFF
37 # quit when 'q' is pressed
38 if key== ord('q'):
39     break
40 if key==ord('i'):
41     inversion_on=not inversion_on
42 if key==ord('h'):
43     hair_on=not hair_on
44
45 # release the cap object
46 vid.release()
47 # destroy all the windows
48 cv2.destroyAllWindows()
```



After-class Reflection

1. Compare binary images, grayscale images, and RGB color images. What information is stored in each type of image, and what information may be lost when moving from color to grayscale or binary?
2. What does a pixel value of 0 mean? What does a value of 255 mean? Why are 256 possible values commonly used in many images?
3. If an image has a very bright red channel but darker green and blue channels, how would you expect the image to look? Explain your reasoning.
4. Why do different image file formats exist? Compare BMP, JPEG, and TIFF in terms of storage size, compression, and possible use cases.
5. Why should we inspect these properties before processing an image? Give an example of a problem that could happen if we ignore them.
6. What kind of visual error might happen if we treat a BGR image as if it were RGB? Why is understanding channel order important?
7. What visual effect do you expect if the red channel of an RGB image is set to zero? How would the result differ if the blue channel were removed instead?



After-class Reflection

8. Give two examples of image filters or effects that could be created by directly modifying pixel values. Explain what numerical operation might be involved.
9. Why does nearest-neighbor interpolation produce blocky artifacts when enlarging an image? In what situation might this simple method still be useful?
10. Compare nearest-neighbor interpolation and bilinear interpolation. Why is bilinear interpolation usually smoother? What might be the disadvantage of using more complex interpolation methods?
11. Why is rotation considered a coordinate transformation? What problems might occur if a rotated pixel location does not correspond exactly to an integer pixel coordinate?
12. Why is it useful to represent geometric transformations using matrices? What is the advantage of combining rotation, translation, and scaling in matrix form?
13. Why can preprocessing steps such as resizing, illumination normalization, color normalization, and alignment affect the performance of a deep learning model?
14. If video processing is image processing repeated frame by frame, what additional challenges might appear when processing videos compared with processing a single image?



Thank You!